

Laboratorio di Calcolo Numerico

Lezione 1

Installare MATLAB

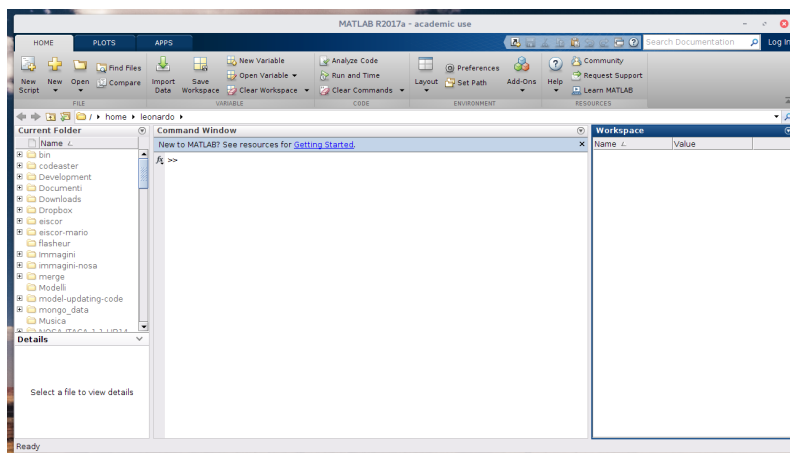
Il primo step per svolgere gli esercizi è avviare MATLAB sul proprio dispositivo, in uno dei seguenti modi:

- Scaricando MATLAB dal sito <https://mathworks.com/>; è possibile registrarsi con le credenziali di Ateneo ed avere accesso ad una copia gratuita di MATLAB per uso personale.
- Utilizzando la versione online di MATLAB, disponibile alla pagina <https://matlab.mathworks.com>. Anche in questo caso, è necessario effettuare il login con le credenziali di Ateneo.
- Installando il software gratuito GNU/Octave dalla pagina <https://www.gnu.org/software/octave/index>, un'alternativa open source a MATLAB. Sebbene non tutte le funzionalità disponibili in MATLAB siano presenti in GNU/Octave, per gli esercizi del laboratorio è possibile utilizzare uno qualunque dei due.

Nel caso si decida di utilizzare MATLAB online, è possibile scaricare i programmi realizzati da drive.matlab.com.

1 Primo programma

Lanciamo MATLAB in una qualunque delle modalità descritte precedentemente. Ci si troverà di fronte ad un'interfaccia simile a questa:



Possiamo scrivere sulla linea di comando, identificata dal cursore >>. Quest'interfaccia è simile al prompt interattivo del terminale.

```
>> 'Hello, world'
ans = Hello, world
```

MATLAB vs Octave: MATLAB è un software proprietario e anche piuttosto costoso! Come studenti potete ottenere una copia “gratuita” tramite la licenza Campus, con cui vi potete esercitare a casa. Conviene però ricordare che esiste un'alternativa non solo gratuita, ma anche Open Source, che si chiama GNU Octave. Purtroppo non è (ancora) all'altezza di MATLAB in tutti gli ambiti, ma dal nostro punto di vista non ci sono differenze. Per cui se volete utilizzare Octave al posto di MATLAB potete farlo senza grosse controindicazioni. Le differenze fra i due sono anche nella sintassi e, sebbene minime, potete controllarle qui: https://en.wikibooks.org/wiki/MATLAB_Programming/Differences_between_Octave_and_MATLAB.

2 Primi calcoli in virgola mobile

MATLAB utilizza la doppia precisione (8 byte per ogni numero). Con i seguenti comandi si possono visualizzare il massimo ed il minimo valore positivo della rappresentazione floating point e la precisione di macchina.

```
>> realmin
ans = 2.2251e-308
>> realmax
ans = 1.7977e+308
>> eps
ans = 2.2204e-16
```

MATLAB come una calcolatrice:

```
>> 1+1
ans = 2
>> 10^10
ans = 1.0000e+10
>> 1e10
ans = 1.0000e+10
>> (1e10)^2
ans = 1.0000e+20
```

Se c'è un punto e virgola alla fine della linea, MATLAB esegue il calcolo ma non scrive il risultato

```
>> 1+1;
>>
```

3 Variabili

Come in ogni linguaggio di programmazione, MATLAB fa utilizzo di variabili, che, in informatica, sono contenitori di dati situate in una porzione della memoria e destinate a contenere valori, che possono (in generale) essere modificati nel corso dell'esecuzione di un programma.

Una variabile è caratterizzata da un nome (inteso solitamente come una sequenza di caratteri e cifre) che deve seguire un insieme di convenzioni che dipende dal linguaggio che si sta adoperando. In MATLAB le seguenti convenzioni devono essere adoperate:

- I nomi delle variabili devono iniziare con una lettera,
- I nomi possono includere ogni combinazione di lettere, numeri, e underscore,
- La lunghezza massima per il nome di una variabile è di 63 caratteri,
- MATLAB è case sensitive. La variabile di nome pAlla è diversa dalla variabile di nome palla.

Inoltre, in MATLAB non è necessaria nè la dichiarazione nè la specifica del tipo di una variabile. Si può infatti procedere direttamente con l'assegnamento ed il sistema assegna o cambia automaticamente il tipo di variabile. Ad esempio

```
a = 100;
```

```
whos a
```

```
Name Size Bytes Class Attributes
```

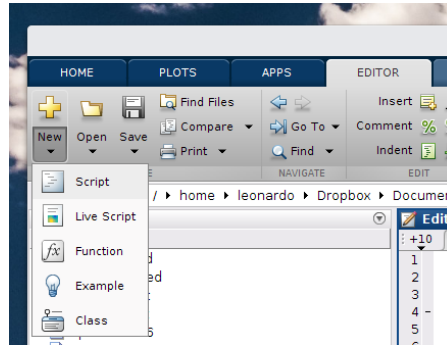
```
a 1x1 8 double
```

4 Creazione di Script

Tutti (o quasi) i comandi visti fino ad ora sono in realtà degli script o delle funzioni pre-costruite e rese disponibili nell'ambiente generale. MATLAB permette all'utente di costruire i suoi script e le sue funzioni per la soluzione di problemi specifici.

Uno script file è semplicemente una collezione di comandi eseguibili di MATLAB e, in alcuni casi più raffinati, di interfacce verso software esterno prodotto in C o in Fortran.

MATLAB dispone di un editor di testo integrato che possiamo usare per questo scopo, cliccando su **New** → **Script** (come mostrato nella figura sotto).



In alternativa si può compilare uno script da un qualsiasi editor di testo (ad esempio gedit). **Per poter eseguire lo script è necessario che questo sia salvato con estensione .m nella cartella da cui è stato lanciato MATLAB.** Per eseguirlo è sufficiente inserire nella linea di comando il nome con cui è stato salvato lo script (senza l'estensione .m).

5 Creazione di Funzioni

Una funzione (detta anche, a seconda del linguaggio di programmazione, routine, subroutine, procedura, metodo), è un particolare costrutto sintattico che raggruppa all'interno di un singolo programma, una sequenza di istruzioni in un unico blocco. Il suo scopo è quello di portare a termine una operazione, azione, o elaborazione in modo tale che, a partire da determinati input, restituisca determinati output.

In MATLAB una funzione di nome `miafunzione` è dichiarata come

```
function [y1,...,yN] = miafunzione(x1,...,xM)
```

che accetta come input `x1,...,xM` e restituisce come output `y1,...,yN`. Questa dichiarazione deve essere la prima istruzione eseguibile del file che contiene la funzione. Nomi validi di funzioni iniziano con un carattere alfabetico e possono contenere lettere, numeri o underscore.

Si può salvare una funzione in:

- un file con estensione .m il cui nome coincida con quella della funzione (es: `miafunzione.m`);
- un file che contiene solamente definizioni di funzione. Il nome del file deve corrispondere esattamente con il nome della prima funzione contenuta nel file;
- uno script che contiene comandi e definizioni di funzione. In questo caso le funzioni devono essere contenute alla fine del file e lo script non può avere lo stesso nome di nessuna delle funzioni contenute al suo interno.

I file possono includere molteplici funzioni locali o innestate. Al fine di produrre del codice leggibile, è consigliato utilizzare sempre la parola chiave `end` per indicare la fine di ogni funzione all'interno di un file.

In particolare, la parola chiave `end` è richiesta ogni qual volta:

- una qualunque funzione in un file contiene una funzione innestata,
- la funzione è una funzione locale all'interno di un file che contiene solo funzioni e, a sua volta, ogni funzione locale usa la parola chiave `end`,
- la funzione è una funzione locale all'intero di uno script.

Per interrompere l'esecuzione di una funzione prima del raggiungimento dell'`end` finale, si può utilizzare la parola chiave `return` che, come suggerisce il nome, restituisce il controllo allo script che ha invocato la funzione. Una chiamata diretta allo script o alla funzione che contiene `return`, non richiama alcun programma di origine e restituisce invece il controllo al prompt dei comandi. Sotto viene riportato come esempio una funzione che calcola l'area del cerchio di raggio fornito dall'utente.

```
function [area] = areacerchio(raggio)
%AREACERCHIO Funzione che calcola l'area del cerchio di raggio r
%se viene fornito un raggio negativo ritorna il valore flag -1
if raggio < 0
    area = -1;
    return
end
area = pi*raggio^2;
end
```

6 Documentazione

Se si incontra una funzione di cui non si conosce/ricorda l'utilizzo è possibile interrogare la guida di MATLAB dalla command window con il comando `help`. Ad esempio, per visualizzare la descrizione relativa alla funzione `cosd` scrivo

```
help cosd
```

che restituisce

```
cosd Cosine of argument in degrees.
cosd(X) is the cosine of the elements of X, expressed in degrees.
For odd integers n, cosd(n*90) is exactly zero, whereas cos(n*pi/2)
reflects the accuracy of the floating point value for pi.
```

```
Class support for input X:
float: double, single
```

See also `acosd`, `cos`.

Documentation for `cosd`

Other functions named `cosd`

7 Perdita di precisione da alcuni calcoli

In generale non tutti i numeri possono essere espressi nel formato floating point, e questo porta ad errori di arrotondamento. Possiamo scegliere ad esempio il numero $\frac{4}{3}$ e provare ad eseguire l'operazione

```
e = 1 - 3*(4/3 - 1)
```

da cui ci aspetteremo di ottenere uno 0, otteniamo invece

```
e = 2.2204e-16
```

che è un errore della grandezza della precisione di macchina.

In maniera analoga, possiamo considerare un altro numero che non è esattamente rappresentato in macchina (ma lo sarebbe in base 10): 0.1. Se lo sommiamo 10 volte in aritmetica esatta ci aspettiamo di ottenere 1:

```
a = 0;
for i = 1:10
    a = a + 0.1;
end
```

Tuttavia se proviamo a fare la verifica leggiamo

```
a == 1
ans =

    logical

    0
```

Ed infatti

```
abs(a-1)
ans = 1.1102e-16
```

Esercizio 1. Scrivere uno script `perditaprec.m` che stampa a schermo gli interi $a \in \{1, \dots, 300\}$ per cui il risultato dell'operazione

```
1 - a*(1/a)
```

non è esattamente 0. Per verificare la corretta implementazione eseguirlo digitando `perditaprec` dal prompt e controllare che si ottenga il seguente output

```
>> perditaprec
k = 49
k = 98
k = 103
k = 107
k = 161
k = 187
k = 196
k = 197
k = 206
k = 214
k = 237
k = 239
k = 249
k = 253
```

Un'altra fonte (più malevola) di errori di arrotondamento si incontra quando si effettuano sottrazioni tra due numeri grossi e molto vicini (*errori di cancellazione*)

```
>> a=1e10
a = 1.0000e+10
>> b=1e4
b = 10000
>> c=(a+b)^2
c = 1.0000e+20
>> format long % scrive piu' cifre
>> c
c = 1.00000200000100e+20
>> c - a^2 - 2*a*b - b^2
ans = 7936
```

Un esempio in cui la cancellazione numerica può essere (parzialmente) evitata è nel calcolo delle radici di un polinomio di secondo grado $ax^2 + bx + c$. La formula risolutiva è infatti

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a},$$

e contempla due potenziali sottrazioni. Quella sotto radice è inevitabile. Quella esterna può essere aggirata calcolando solo la soluzione in cui i segni dei due addendi sono concordi e ricavandosi l'altra sfruttando la relazione

$$x_1 \cdot x_2 = \frac{c}{a}.$$

Esercizio 2. Scrivere una funzione

```
function [x1, x2] = solve2(a,b,c)
```

che risolva l'equazione di secondo grado $ax^2 + bx + c = 0$ nel modo più stabile possibile, ovvero evitando la sottrazione più esterna. Testare l'implementazione su $x^2 - 10^{10}x +$

$10^{-10} = 0$, confrontando i risultati con quelli ottenuti tramite l'applicazione della formula standard (quindi con cancellazione). Come soluzione corretta di riferimento si può lanciare il comando

```
roots([a b c])
```

8 Approssimazione dell'esponenziale

Per funzioni non razionali, come ad esempio e^x , ci si riconduce ad approssimarne i valori tramite espansioni polinomiali o razionali (rapporto di polinomi). Nel caso dell'esponenziale possiamo ad esempio considerare il polinomio di Taylor di un certo grado, ottenuto tramite lo sviluppo di Taylor nel punto 0.

```
function a=myexp(x,n)
% calcola exp(x) con la serie di Taylor troncata all'n-esimo termine
a=1; %accumulatore
for k=1:n
    a=a+x^k/factorial(k);
end
end
```

Qualche esperimento su quanti termini servono per approssimare bene — confrontiamo con la funzione `exp(x)` di MATLAB, che calcola l'esponenziale meglio di noi.

```
>> myexp(1,5)
ans = 2.7167
>> myexp(10,50)
ans = 2.2026e+04
>> exp(10)
ans = 2.2026e+04
>> format long
>> myexp(10,50)
ans = 22026.4657948067
>> exp(10)
ans = 22026.4657948067
```

Due problemi:

- Inaccurato: prova `myexp(-20,500)`
- Lento: con n termini, il numero di operazioni da eseguire cresce come n^2 (perché?)

Risolviamo (2) introducendo un altro accumulatore:

```
function a=myexp2(x,n)
%calcola e^x con Taylor troncato
%ma usa solo O(n) operazioni
```



```

t = 1; %accumulatore che contiene il termine generico della sommatoria
a = 1; %accumulatore che contiene le somme parziali
for k = 1 : n
    t = t * x / k;
    a = a + t;
end
end

```

Ora va meglio:

```

>> myexp(-20,500)
ans = NaN
>> myexp2(-20,500)
ans = 5.62188447213042e-09

```

Cosa succedeva?

```

>> factorial(500)
ans = Inf
>> -20^500
ans = -Inf
>> -Inf/Inf
ans = NaN

```

Ci sono ancora problemi di accuratezza sui numeri negativi:

```

>> myexp2(-30,500)
ans = -3.06681235635622e-05

```

Un esponenziale dovrebbe sempre essere positivo... Ci sono pesanti errori di cancellazione nella formula che abbiamo usato (perché?).

La soluzione: cambiare algoritmo e sceglierne uno che non porti a errori di cancellazione, ad esempio possiamo osservare che

```

>> exp(-30)
ans = 9.3576e-14
>> myexp2(-30,500)
ans = -3.0668e-05
>> 1/myexp2(30,500)
ans = 9.3576e-14
>> format long
>> exp(-30)
ans = 9.35762296884017e-14
>> 1/myexp2(30,500)
ans = 9.35762296884017e-14

```

Esercizio 3. Scrivere una funzione `myexp3(x)` che controlla se x è negativo o positivo, e a seconda del segno calcola l'esponenziale come $1/e^{-x}$ oppure e^x con la serie di Taylor troncata a $n = 500$.

Laboratorio di Calcolo Numerico

Lezione 2

1 Vettori e matrici

MATLAB è pensato per lavorare con vettori e matrici; pertanto, ha una sintassi specifica e parecchi comandi dedicati, che rendono molto più semplice lavorare con i vettori rispetto ad un linguaggio generico come il C.

1.1 Creare vettori e matrici

```
>> A=[1 2 3; 4 5 6]
A =

1 2 3
4 5 6
```

```
>> zeros(3,2)
ans =

0 0
0 0
0 0
```

```
>> ones(3,2)
ans =

1 1
1 1
1 1
```

```
>> eye(3)
ans =

1 0 0
```

```

0 1 0
0 0 1

>> randn(2,3)
ans =

0.5377 -2.2588 0.3188
1.8339 0.8622 -1.3077

```

1.2 Il *range operator* :

Con la sintassi `a:t:b` creiamo un vettore (riga) che contiene gli elementi `a`, `a+t`, `a+2t`, `a+3t`, ... fino a `b` (o fino all'ultimo che sia minore o uguale a `b`). Se `t=1`, si può semplicemente usare il comando `a:b`

```

>> 1:0.5:4
ans =

1.0000 1.5000 2.0000 2.5000 3.0000 3.5000 4.0000

>> 1:10
ans =

1 2 3 4 5 6 7 8 9 10

>> 1:2:10
ans =

1 3 5 7 9

```

Dove avete già usato l'operatore `:`?

1.3 Il comando `linspace`

Un'utile variante dell'operatore `:` è il comando `linspace` che, come il nome suggerisce, permette di dividere un intervallo in parti uguali. Supponiamo ad esempio di voler dividere $[0, 2\pi]$ in $n - 1$ sotto-intervalli di eguale lunghezza. Possiamo dare il comando `t = linspace(0, 2*pi, n)` ed ottenere un vettore `t` tale che

- $t(1) = 0$
- $t(n) = 2\pi$
- Per ogni i fra 2 ed n , $t(i) - t(i-1) = 2\pi / (n-1)$.

Spesso `linspace` è il modo più naturale per suddividere un intervallo su cui si desidera plottare una funzione. Utilizzate `help linspace` per approfondire l'utilizzo del comando.

1.4 Accedere agli elementi

```
>> A=ones(2,3)
A =

1 1 1
1 1 1

>> A(1,2)=2
A =

1 2 1
1 1 1

>> A(1,2)
ans = 2

>> A(5,10)
??? Index exceeds matrix dimensions.

>> A(5,10)=7
A =

1 2 1 0 0 0 0 0 0 0
1 1 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 7
```

Se cerco di *leggere* un elemento che non esiste (perché la matrice è troppo piccola), ottengo un errore. Se cerco di *scrivere* un elemento che non esiste, la matrice viene automaticamente ingrandita.

1.5 Operazioni su vettori

```
>> a=1:3
a =

1 2 3

>> b=4:6
b =

4 5 6
```

```

>> a+b
ans =

5 7 9

>> sin(a)
ans =

0.8415 0.9093 0.1411

>> 2*a+1
ans =

3 5 7

>> a.*b % operazioni elemento per elemento
ans =

4 10 18

>> c=a' % matrice trasposta
c =

1
2
3

>> C=a'*b % prodotto matrice-matrice
C =

4 5 6
8 10 12
12 15 18

>> length(a) % lunghezza di un vettore
ans = 3

>> size(C) % dimensioni di una matrice - (righe, colonne)
ans =

3 3

```

1.6 Costruire una matrice per diagonali

È possibile inserire direttamente le entrate di una matrice che giacciono su una certa diagonale con il comando `diag`. Più precisamente digitando `A = diag(v,k)` si ottiene una matrice che ha gli elementi del vettore `v` sulla `k`-esima diagonale e 0 altrove. Il segno negativo o positivo di `k` corrisponde alle sottodiagonali e alle sopradiagonali, rispettivamente. Se viene omesso il parametro `k`, gli elementi di `v` vengono inseriti sulla diagonale principale. In questo modo risulta facilitata l'inizializzazione di matrici con struttura a banda: bidiagonali, tridiagonali, ecc.

```
>> A=diag(1:10)+diag(ones(9,1),1)
A =

1 1 0 0 0 0 0 0 0 0
0 2 1 0 0 0 0 0 0 0
0 0 3 1 0 0 0 0 0 0
0 0 0 4 1 0 0 0 0 0
0 0 0 0 5 1 0 0 0 0
0 0 0 0 0 6 1 0 0 0
0 0 0 0 0 0 7 1 0 0
0 0 0 0 0 0 0 8 1 0
0 0 0 0 0 0 0 0 9 1
0 0 0 0 0 0 0 0 0 10
```

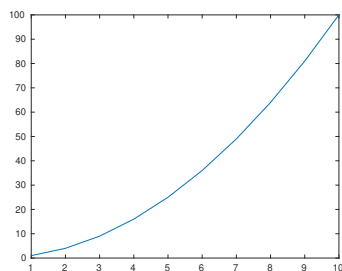
2 Grafici

Il comando `plot(x,y)` prende come argomenti due vettori della stessa lunghezza `x` e `y` e disegna sul piano cartesiano i punti `x(i),y(i)` collegandoli con una linea.

```
>> r=1:10
r =

1 2 3 4 5 6 7 8 9 10

>> plot(r,r.^2)
```



Il seguente comando disegna un cerchio.

```
>> t=0:.001:2*pi;  
>> plot(cos(t),sin(t))
```

Suggerimento: Il comando `t = 0:.001:2*pi` può essere sostituito da un comando `linspace`. Provate ad utilizzarlo nei prossimi esercizi.

Esercizio 1. Scrivere una funzione `circle(z,r)` che, dato un complesso `z` e un reale $r \geq 0$, disegni il cerchio di centro `(real(z),imag(z))` e raggio `r`. Testare con `circle(1+2*i,2)`. A quale ascissa/ordinata arriva il cerchio?

3 Sottomatrici e determinanti

Utilizzando l'operatore `:` (*range operator*), in MATLAB è possibile selezionare un'intera sottomatrice di una matrice:

```
>> A=[1 2 3; 4 5 6; 7 8 9]  
A =  
  
1 2 3  
4 5 6  
7 8 9  
  
>> A(1:2,2:3)  
ans =  
  
2 3  
5 6  
  
>> A(2:end,1:end-2)  
ans =  
  
4  
7  
  
>> A(1:end,1:end)  
ans =  
  
1 2 3  
4 5 6  
7 8 9  
  
>> A(1,:)   
ans =  
  
1 2 3
```

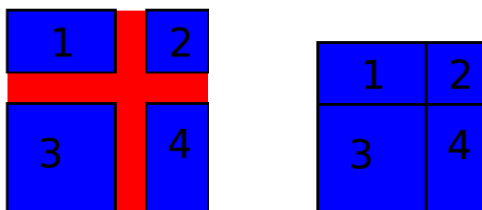
La sintassi `a:b` seleziona tutte le righe/colonne comprese tra `a` e `b` (estremi inclusi). Il valore `end` viene sostituito con il massimo indice disponibile. Il solo `:` è un'abbreviazione per `1:end`.

Possiamo anche assegnare un valore a una sottomatrice selezionata in questo modo:

```
>> A(1:2,1:2)=eye(2)
A =
1 0 3
0 1 6
7 8 9
```

Ovviamente le dimensioni devono essere compatibili: non posso selezionare una sottomatrice 2×2 e assegnarle il valore `eye(3)`!

La seguente function restituisce la sottomatrice corrispondente al *minore* (i, j) in A , cioè la matrice che si ottiene eliminando la i -esima riga e la j -esima colonna di A .



```
function B = submatrix_minor(A,i,j)
X=A(1:i-1,1:j-1);
Y=A(1:i-1,j+1:end);
Z=A(i+1:end,1:j-1);
W=A(i+1:end,j+1:end);
B=[X Y; Z W];
end
```

Abbiamo già visto che se X, Y, Z, W sono numeri, la riga di codice `B=[X Y; Z W]` crea la matrice

$$\begin{bmatrix} X & Y \\ Z & W \end{bmatrix}.$$

Ora vediamo che la stessa sintassi funziona anche se X, Y, Z, W sono matrici di dimensioni "compatibili" e crea la matrice formata accostando i quattro blocchi.

Esercizio 2. Creare una `function` `d=mydet(A)` che calcoli il determinante di una matrice quadrata A utilizzando la formula di Laplace sulla prima riga, cioè

$$\det(A) = A_{11} \det A^{(11)} - A_{12} \det A^{(12)} + A_{13} \det A^{(13)} - \dots + (-1)^{n+1} A_{1n} \det A^{(1n)}$$

dove A_{ij} è l'elemento di A nella posizione (i, j) e $A^{(ij)}$ è la matrice minore di A rispetto a (i, j) . [Hint: la funzione può essere *ricorsiva*, cioè chiamare se stessa al suo interno. Fate attenzione: bisogna definire un caso base!]

Poi testarla su alcune matrici, confrontandola con la funzione `det` di MATLAB, per esempio matrici con elementi casuali `V=randn(2)`, `V=randn(3)`, `V=randn(4)`...

Esercizio 3. La seguente function è equivalente a `submatrix_minor(A,i,j)`. Come funziona? [Hint: osservate cosa succede eseguendo `A(v,w)` dove `v` e `w` sono vettori contenenti interi.]

```
function B = submatrix_minor2(A,i,j)
B = A([1:i-1 i+1:size(A,1)], [1:j-1 j+1:size(A,2)]);
end
```

4 Tempi di calcolo

Le funzioni `tic` e `toc` misurano il tempo necessario ad eseguire più istruzioni. La prima fa partire il cronometro, la seconda lo ferma e restituisce il valore ottenuto. Per esempio, le istruzioni

```
tic;
A=randn(100);
B=randn(100);
A*B;
t=toc;
```

salvano in `t` il tempo necessario ad eseguire le due righe centrali.

La seguente funzione disegna un grafico del tempo impiegato per calcolare il prodotto di matrici $n \times n$, generate casualmente, con n crescente.

```
function plottimes();
n = [100, 200, 400, 800, 1600, 3200, 6400];
nn = length(n);
tempi=zeros(nn,1); %prepara un vettore vuoto con i tempi
for i=1:nn
    A=randn(n(i)); %le matrici vengono generate prima di 'tic'
    B=randn(n(i));
    tic;
    C=A*B;
    tempi(i)=toc;
end
plot(n,tempi);
end
```

I tempi di calcolo dovrebbero crescere come $\mathcal{O}(n^3)$, in particolare

$$\text{tempo}(2n) \approx 8 \cdot \text{tempo}(n).$$

Se mostrato in un grafico con scala logaritmica sugli assi x e y , ad esempio usando il comando

```
loglog(n, tempi)
```

l'andamento dovrebbe essere quello di una retta con coefficiente angolare 3 ($y = 3x + k$).

Esercizio 4. Calcolare i tempi di esecuzione del prodotto $A*B$ e della somma $A+B$ di matrici $n \times n$ per i valori crescenti di n considerati nell'esempio sopra. Mostrare i tempi su un grafico che utilizzi la scala logaritmica per entrambi gli assi (usare il comando `loglog` invece di `plot`), in modo da poter confrontare meglio tempi su scale molto diverse.

Per riferimento riportare anche i grafici di $y = n^3$ e $y = n^2$, per verificare che le pendenze delle rette rappresentanti i costi della moltiplicazione e n^3 coincidano, almeno asintoticamente, così come quelli della somma e della funzione n^2 .

Per valori piccoli di n , ad esempio 100, per cui i tempi di esecuzione sono molto piccoli, ci possono essere delle oscillazioni nei tempi dovuti al fatto che MATLAB è un linguaggio interpretato. Nel caso si vogliano delle misurazioni più affidabili si può utilizzare la funzione `timeit` (si digiti `help timeit` per vedere come funziona) che fa una media dei tempi su un certo numero di esecuzioni.

Laboratorio di Calcolo Numerico

Lezione 3

1 Disegnare un cerchio

Il seguente comando disegna il cerchio di centro $(0,0)$ e raggio 1.

```
>> t=0:.001:2*pi;  
>> plot(cos(t),sin(t))
```

Suggerimento: Il comando `t = 0:.001:2*pi` può essere sostituito da un comando `linspace`. Provate ad utilizzarlo nei prossimi esercizi.

Esercizio 1. Scrivere una funzione `circle(z,r)` che, dato un complesso z e un reale $r \geq 0$, disegni il cerchio di centro $(\text{real}(z), \text{imag}(z))$ e raggio r . Testare con `circle(1+2*i,2)`.

Il disegno del cerchio può essere un poco più elaborato scegliendo un colore per il bordo e per la parte interna. A tal proposito si può utilizzare la funzione `patch` (per vedere come funziona digitare `help patch`).

2 Cerchi di Gerschgorin

La seguente funzione disegna gli autovalori e i cerchi di Gerschgorin di una matrice quadrata A .

```
function disegna_gersh(A)  
n=size(A, 1); %estrae il numero di righe di A  
  
close all %elimina i disegni preesistenti  
hold on %fa sì che ogni disegno non cancelli il precedente  
axis('equal') %forza la stessa scala su x e y  
  
autovalori=eig(A);  
plot(real(autovalori),imag(autovalori),'*');  
% '*': disegna solo i punti, non collegandoli con linee  
% 'help plot' per altre stringhe magiche  
for k=1:n
```

```

        center=A(k,k);
        radius=0; %accumulatore
        for j=1:n
            if(j~=k)
                radius=radius+abs(A(k,j));
            end
        end
    end
    circle(center,radius);
end
end
end

```

Esercizio 2. Modificare la funzioni `disegna_gersh` e `circle` in modo che disegnino cerchi di un colore assegnato dall'utente e si testino su alcune matrici, ad esempio: `rand(5)`, `randn(5)`, `hilb(5)`.

Nel caso si sia utilizzato un colore per la parte interna dei cerchi, si può utilizzare il comando `alpha(x)` per regolarne l'intensità; ad esempio si può dare il comando:

```
alpha(.15)
```

quando tutti i cerchi sono stati aggiunti alla figura.

Esercizio 3. Si provi a riscrivere `disegna_gersh` con un unico ciclo `for`. Infatti, poiché MATLAB è un linguaggio *interpretato*, eseguire ogni singola istruzione ha un “costo” non trascurabile (il computer deve leggere la riga, interpretarla e trasformarla in codice macchina). Per questo se si riesce a riscrivere le funzioni utilizzando delle operazioni sui vettori invece che dei cicli `for`, il programma gira molto più velocemente. Per esempio, è molto più veloce

```
s=sum(abs(v));
```

(una istruzione da interpretare) rispetto a

```

s=0;
for k=1:length(v)
    s=s+abs(v(k));
end

```

($O(n)$ istruzioni da interpretare, dove n è la lunghezza del vettore v). Negli anni recenti, MATLAB ha incorporato un compilatore JIT che ha permesso di accelerare i `for` più semplici come quello riportato sopra. Tuttavia, questa osservazione sarà importante nel caso di `for` annidati per il calcolo delle fattorizzazioni matriciali, che vedremo nelle prossime lezioni.

Esercizio 4. Si scriva una funzione `disegna_gersh2` che data una matrice A disegni in una figura: i suoi autovalori, i cerchi di gershgorin della matrice A **in blu** ed **in giallo** quelli associati alla matrice A^T ; in questo modo si dovrebbe poter visualizzare l'insieme

$$\left(\bigcup_{i=1}^n F_i(A) \right) \cap \left(\bigcup_{i=1}^n F_i(A^T) \right)$$

come l'intersezione delle due regioni colorate.

Laboratorio di Calcolo Numerico

Lezione 4

1 Matrici di permutazione e riducibilità

Ogni permutazione sull'insieme $\{1, \dots, n\}$ si può rappresentare tramite un vettore $p \in \mathbb{N}^n$ (fatto con i numeri interi da 1 a n nell'ordine che segue la permutazione). Ad esempio, nel caso $n = 4$, per rappresentare la permutazione che manda $j + 1$ in j e 1 in 4 avremo

```
p = [2 3 4 1];
```

Sia $\Pi \in \mathbb{R}^n$ un certa matrice di permutazione ottenuta applicando la permutazione p alle righe dell'identità. In Matlab moltiplicare una matrice A a sinistra per Π è particolarmente semplice, in quanto basta applicare la permutazione direttamente agli indici di riga di A . Ad esempio per calcolare $\Pi \cdot A$ si può procedere come segue

```
A = [ 1 2 3 4; 5 6 7 8; 9 10 11 12; 13 14 15 16];
```

```
>> A
```

```
A =
```

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
```

```
A(p, :)
```

```
ans =
```

```
5 6 7 8
9 10 11 12
13 14 15 16
1 2 3 4
```

Moltiplicare a destra per Π (ovvero calcolare $A \cdot \Pi$) è un filo più complicato perchè la permutazione Π rappresenta, se vista per colonne, la permutazione inversa rispetto a p . Quindi è necessario trovare la permutazione inversa di p ; questo si può fare come segue

```
n = length(p);
ip = zeros(1, n); % calcolo la permutazione inversa come quella
ip(p) = 1:n;      % che composta con p da 1:n
A(:, ip)

ans =

4 1 2 3
8 5 6 7
12 9 10 11
16 13 14 15
```

Infine, dato che Π^T rappresenta la permutazione inversa rispetto a Π , si ha che $\Pi A \Pi^T$ si trova semplicemente con il comando

```
A(p, p)

ans =

6 7 8 5
10 11 12 9
14 15 16 13
2 3 4 1
```

Esercizio 1. Sia

$$A = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 2 & 5 & 0 & -3 \\ 0 & 1 & 3 & 6 \\ 2 & 0 & 0 & -1 \end{bmatrix}.$$

si trovi una matrice di permutazione Π tale per cui $\Pi A \Pi^T$ risulta triangolare superiore a blocchi. Si verifichi la correttezza del risultato trovato con Matlab.

2 Soluzione di sistemi triangolari

Consideriamo il seguente sistema triangolare inferiore

$$\begin{pmatrix} L_{11} & 0 & 0 & 0 & 0 \\ L_{21} & L_{22} & 0 & 0 & 0 \\ L_{31} & L_{32} & L_{33} & 0 & 0 \\ L_{41} & L_{42} & L_{43} & L_{44} & 0 \\ L_{51} & L_{52} & L_{53} & L_{54} & L_{55} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \end{pmatrix}.$$

Possiamo risolverlo per sostituzione: a ogni passo $i = 1, \dots, n$, supponendo di avere calcolato $x_1, \dots, x_{i-1}$, si ha

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} L_{ij}x_j}{L_{ii}}.$$

In particolare $x_1 = \frac{b_1}{L_{11}}$.

```
function x=inf_solve(L,b)
s=size(L);
n=s(1);
x=zeros(n,1);
for i=1:n
    p=b(i); % accumulatore
    for j=1:i-1
        p=p-L(i,j)*x(j);
    end
    x(i)=p/L(i,i);
end
end
```

Il comando `tril` estrae la parte triangolare inferiore di una matrice fornita come input; analogamente `triu` estrae la parte triangolare superiore. Testiamo la funzione `inf_solve` con $L=\text{tril}(\text{randn}(5))$ e termine noto $L*y$, dove $y=[1:5]'$ (in questo modo sappiamo che la soluzione del sistema è proprio y).

```
>> y=[1:5]';
y =
1
2
3
4
5

>> L=tril(randn(5))
L =

0.5377    0         0         0         0
1.8339 -0.4336    0         0         0
-2.2588 0.3426 0.7254    0         0
0.8622 3.5784 -0.0631 1.4090    0
0.3188 2.7694 0.7147 1.4172 0.4889

>> inf_solve(L,L*y)
ans =
```

```

1.0000
2.0000
3.0000
4.0000
5.0000

```

Esercizio 2. Scrivere una **function** `sup_solve(U,b)` che risolva un sistema $Ux = b$ con U triangolare superiore (suggerimento: sostituire a partire dall'ultima riga). Testare su $U=\text{triu}(\text{randn}(5))$, $b=U*y$ (con y vettore scelto).

Esercizio 3. Modificare `inf_solve` e `sup_solve` in modo da sostituire il ciclo **for** più interno con operazioni su vettori. Chiamate queste nuove funzioni rispettivamente `inf_solve2` e `sup_solve2` e confrontate le performance su matrici grandi (ad esempio 1000×1000).

3 Eliminazione di Gauss

Ricordate come funziona il passo k -esimo dell'algoritmo di eliminazione di Gauss (nell'immagine $k = 3$):

$$\left(A^{(k)} \mid b^{(k)} \right) = \left(\begin{array}{cccccc|c} * & * & * & * & * & * & b_1^{(1)} \\ 0 & * & * & * & * & * & b_2^{(2)} \\ 0 & 0 & A_{33}^{(k)} & * & * & * & b_3^{(3)} \\ 0 & 0 & A_{43}^{(k)} & * & * & * & b_4^{(3)} \\ 0 & 0 & A_{53}^{(k)} & * & * & * & b_5^{(3)} \\ 0 & 0 & A_{63}^{(k)} & * & * & * & b_6^{(3)} \end{array} \right)$$

dovete modificare $A^{(k)}|b^{(k)}$ sommando alle sue righe, da $k+1$ ad n , multipli della riga k in modo che gli elementi in posizione $(k+1, k), (k+2, k), \dots, (n, k)$ diventino zero.

Esercizio 4. Scrivete una **function** `[U, c] = my_gauss(A, b)` che applichi l'eliminazione di Gauss al sistema lineare $Ax = b$ restituendo la matrice in forma triangolare U ed il vettore dei termini noti c , tale che il sistema $Ux = c$ sia equivalente a quello di partenza.

Esercizio 5. Scrivete una **function** `x = sys_solve(A,b)` che risolva un sistema lineare generico utilizzando l'eliminazione di Gauss, ed il metodo implementato in `sup_solve`. Si testi il metodo usando matrici e vettori generati casualmente con la function handle `randn`, ad esempio eseguendo:

```

n=5;
A=randn(n);
x=randn(n, 1);
b=A*x;
y= sys_solve(A,b);
x-y

```


In caso di implementazione corretta ci aspettiamo che il vettore $x - y$ mostrato a schermo abbia tutte le entrate vicine a 0.

Laboratorio di Calcolo Numerico

Lezione 5

1 Sistemi con più termini noti

Si può utilizzare l'eliminazione di Gauss anche per risolvere un'equazione della forma

$$AX = B, \quad B \in \mathbb{C}^{n \times s}, \quad X \in \mathbb{C}^{n \times s},$$

che è equivalente al risolvere s sistemi lineari

$$Ax^{(1)} = b^{(1)}, \quad Ax^{(2)} = b^{(2)}, \quad \dots \quad Ax^{(s)} = b^{(s)},$$

in cui $x^{(j)}, b^{(j)}$ rappresentano le colonne di X e B rispettivamente. Quello che si deve fare per applicare l'eliminazione di Gauss in questo contesto è applicare la fase di triangolarizzazione alla matrice aumentata $(A \mid B)$ ed infine risolvere gli s sistemi triangolari trovati (che condividono la stessa matrice dei coefficienti).

Ad esempio per

$$A = \begin{bmatrix} -2 & 1 & 1 \\ -1 & 3 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} -2 & 0 \\ -3 & 0 \\ 2 & 1 \end{bmatrix}$$

si ha nella prima fase

$$\left[\begin{array}{ccc|cc} -2 & 1 & 1 & -2 & 0 \\ -1 & -3 & 1 & -3 & 0 \\ 1 & 1 & 2 & 2 & 1 \end{array} \right] \rightarrow \left[\begin{array}{ccc|cc} -2 & 1 & 1 & -2 & 0 \\ 0 & \frac{5}{2} & \frac{1}{2} & -2 & 0 \\ 0 & \frac{3}{2} & \frac{5}{2} & 1 & 1 \end{array} \right] \rightarrow \left[\begin{array}{ccc|cc} -2 & 1 & 1 & -2 & 0 \\ 0 & \frac{5}{2} & \frac{1}{2} & -2 & 0 \\ 0 & 0 & \frac{11}{5} & \frac{11}{5} & 1 \end{array} \right]$$

ed infine la risoluzione di

$$\begin{bmatrix} -2 & 1 & 1 \\ 0 & \frac{5}{2} & \frac{1}{2} \\ 0 & 0 & \frac{11}{5} \end{bmatrix} x^{(1)} = \begin{bmatrix} -2 \\ -2 \\ \frac{11}{5} \end{bmatrix} \Rightarrow x^{(1)} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & 1 & 1 \\ 0 & \frac{5}{2} & \frac{1}{2} \\ 0 & 0 & \frac{11}{5} \end{bmatrix} x^{(2)} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \Rightarrow x^{(2)} = \begin{bmatrix} \frac{2}{11} \\ -\frac{1}{11} \\ \frac{5}{11} \end{bmatrix}$$

che ci dice che la soluzione è

$$X = \begin{bmatrix} 1 & \frac{2}{11} \\ -1 & -\frac{1}{11} \\ 1 & \frac{5}{11} \end{bmatrix}.$$

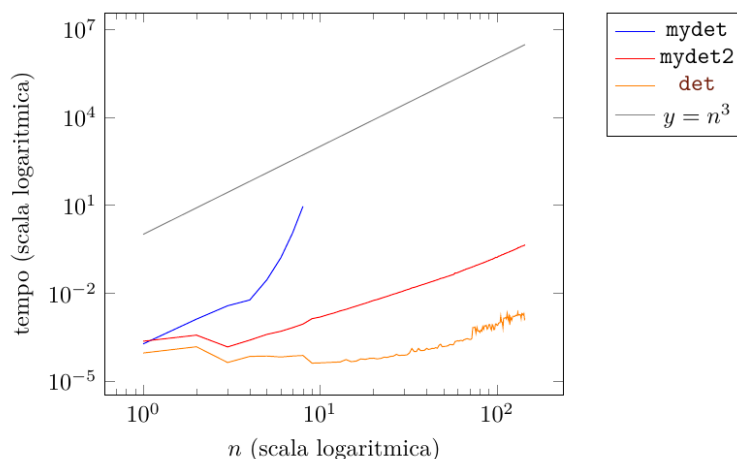
Esercizio 1. Modificare la funzione `sys_solve` implementata nella lezione 4, in modo che gestisca il caso di un termine noto B con più di una colonna, risolvendo l'equazione $AX = B$ (quindi ritornando una soluzione con più colonne) tramite l'eliminazione di Gauss generalizzata descritta sopra. **Suggerimento:** per risolvere i sistemi triangolari alla fine si può eseguire un for loop in cui viene chiamata la funzione `sup_solve` implementata nella lezione 4.

2 Determinante con l'eliminazione di Gauss

Esercizio 2. Scrivete una **function** `d=mydet2(A)` che calcoli il determinante utilizzando l'eliminazione di Gauss. Suggerimenti:

- utilizzate espressioni del tipo `A(i,:)=A(i,:)+A(j,:)` per sommare due righe di una matrice, invece di scrivere un ciclo `for`.
- per ora, ignorate il fatto che i pivot possono essere zero; se incontrate un pivot nullo, terminate con un messaggio di errore (potete farlo con l'istruzione `error('ho incontrato un pivot nullo')`).
- per controllare che tutto vada bene, in una prima fase fatevi scrivere a schermo il valore di A dopo ogni passo di eliminazione di Gauss

Esercizio 3. Come nell'esercizio 4 della lezione 2, disegnare un grafico in cui si riportano i tempi di calcolo di `mydet2` per matrici $n \times n$ (generate in maniera casuale) per n che cresce da 1 a 150; si utilizzi la scala logaritmica per entrambi gli assi (usare il comando `loglog` invece di `plot`), in modo da poter confrontare meglio tempi su scale molto diverse. Si aggiungano i tempi di calcolo della funzione `mydet` (implementata nella lezione 2 e per cui ci si può fermare per $n = 10$) e di `det` che è la funzione di Matlab per il calcolo del determinante. Il grafico dovrebbe assomigliare a questo:



Per riferimento abbiamo riportato anche il grafico di $y = n^3$, sempre sulla stessa scala logaritmica. Qual è la pendenza della retta che corrisponde ad esso? Come potete determinare dal grafico che `mydet2` richiede $O(n^3)$ operazioni mentre `mydet` è asintoticamente più costoso? La funzione `det` di MATLAB dovrebbe essere anch'essa cubica, ma il suo comportamento è più complicato da determinare per via delle ottimizzazioni usate; i tempi esatti dipendono fortemente dall'architettura del calcolatore.

3 Metodi di Jacobi e Gauss–Seidel

3.1 Matrici di test

Inserire in MATLAB le seguenti matrici.

$$A1 = \begin{bmatrix} 3 & 0 & 4 \\ 7 & 4 & 5 \\ -1 & -2 & 2 \end{bmatrix}, \quad A2 = \begin{bmatrix} -3 & 3 & -6 \\ -4 & 7 & -8 \\ 5 & 7 & -9 \end{bmatrix},$$

$$A3 = \begin{bmatrix} 4 & 1 & 1 \\ 2 & -9 & 0 \\ 0 & -8 & -6 \end{bmatrix}, \quad A4 = \begin{bmatrix} 7 & 6 & 9 \\ 4 & 5 & -4 \\ -7 & -3 & 8 \end{bmatrix}.$$

Vogliamo verificare che

- Su $A1$, il metodo di Jacobi non converge ma quello di Gauss–Seidel sì;
- Su $A2$, Jacobi converge (piano) ma Gauss–Seidel no;
- Su $A3$, convergono entrambi e Gauss–Seidel è più veloce;
- Su $A4$, convergono entrambi (piano) e Jacobi è più veloce.

3.2 Il metodo di Jacobi

La formula che definisce il metodo di Jacobi per la risoluzione del sistema $Ax = b$ è

$$x_i^{(new)} = \frac{1}{A_{ii}} \left(b_i - \sum_{j=1}^{i-1} A_{ij} x_j^{(old)} - \sum_{j=i+1}^n A_{ij} x_j^{(old)} \right), \quad i = 1, \dots, n.$$

Esercizio 4. Scrivere una funzione `function x=jacobi(A,b,k)` che esegue k passi del metodo di Jacobi. Dopo ogni passo, scrivere sullo schermo la norma-2 del *residuo* con l'istruzione `norm(A*x-b)` (senza punto e virgola finale).

Suggerimento: attenzione che non potete “riutilizzare” la variabile x in un ciclo del tipo

```
for i=1:n
...
x(i) = (formule che coinvolgono elementi di x);
...
end
```

perché in questo modo dopo la prima iterazione il vecchio valore di $x(1)$ va perso, ma a voi serve ancora nelle formule che calcolano i nuovi $x(2), x(3), \dots$. Dovrete invece fare qualcosa del tipo

```
x_new=zeros(n,1);
for i=1:n
...
x_new(i) = (formule che coinvolgono elementi di x);
...
end
x=x_new;
```

Testare la funzione sulle quattro matrici di test $A1, \dots, A4$, con termine noto b a piacere (per esempio il vettore con tutti gli elementi uguali a 1) e vettore iniziale $x^{(0)} = b$: dovrebbe venire qualcosa simile all'output qui sotto. I numeri precisi calcolati da voi possono cambiare (dipende da un paio di scelte che potete fare nell'algoritmo), però il comportamento della successione (va a zero? Quanto velocemente?) dovrebbe corrispondere.

```
>> jacobi(A1,ones(3,1),10)
ans = 13.6839
ans = 38.2823
ans = 57.3885
ans = 68.4757
ans = 189.6514
ans = 136.0946
ans = 488.7049
ans = 571.4353
ans = 977.2891
ans = 2.1035e+03

ans =

149.5063
218.0709
9.7088

>> jacobi(A2,ones(3,1),10)
ans = 17.088015
ans = 26.818686
ans = 53.445524
ans = 93.296349
ans = 177.191043
ans = 320.662796
ans = 597.366966
ans = 1094.297461
```

```

ans = 2024.546226
ans = 3724.075589

ans =

574.9793
387.4866
337.3805

>> jacobi(A3,ones(3,1),10)
ans = 7.1111
ans = 2.2222
ans = 1.5062
ans = 0.40329
ans = 0.24829
ans = 0.13397
ans = 0.029064
ans = 0.025834
ans = 0.010817
ans = 0.0028290

ans =

0.287301
-0.046952
-0.104023

>> jacobi(A4,ones(3,1),10)
ans = 23.400
ans = 12.514
ans = 4.3015
ans = 3.6876
ans = 2.8242
ans = 1.0743
ans = 0.75329
ans = 0.68841
ans = 0.29360
ans = 0.25730

ans =

-0.261799
0.430087
0.056597

```

I risultati non sono molto significativi perché 10 iterazioni sono molto poche, provare per esempio con $k = 50$. Quando c'è convergenza? Quante iterazioni servono perché il residuo scenda sotto la soglia $\varepsilon = 10^{-8}$?

Notate che è possibile implementare il metodo di Jacobi con un solo ciclo **for** sulle iterazioni del metodo. Come?

3.3 Il metodo di Gauss–Seidel

La formula che definisce il metodo di Gauss–Seidel per la risoluzione del sistema $Ax = b$ è invece

$$x_i^{(new)} = \frac{1}{A_{ii}} \left(b_i - \sum_{j=1}^{i-1} A_{ij} x_j^{(new)} - \sum_{j=i+1}^n A_{ij} x_j^{(old)} \right), \quad i = 1, \dots, n.$$

Notate che l'unica cosa che cambia rispetto a Jacobi è il *(new)* in rosso.

Esercizio 5. Scrivere una funzione **function** `x=gaussseidel(A,b,k)` che esegua k passi del metodo di Gauss–Seidel. Dopo ogni passo, scrivere sullo schermo la norma-2 del residuo con l'istruzione **norm**(`A*x-b`) (senza punto e virgola finale).

Suggerimento: come prima, aggiornate gli elementi $1, 2, 3, \dots$ in quest'ordine: ad ogni passo i , nell'equazione che corrisponde alla riga i -esima della matrice c'è solo un elemento incognito $x_i^{(new)}$, quindi potete calcolarlo risolvendo l'equazione. In questo modo evitate di dover risolvere il sistema triangolare superiore con **sup_solve**...

Testare il metodo sulle quattro matrici di test. Quante iterazioni servono per ognuna delle matrici perché il residuo scenda sotto $\varepsilon = 10^{-8}$?

Laboratorio di Calcolo Numerico

Lezione 5

Trasformazioni di Householder

Una trasformazione di Householder su $\mathbb{R}^m$ è una matrice ortogonale della forma $H_v = I - \frac{2vv^T}{\|v\|_2^2}$ per un certo vettore $v \in \mathbb{R}^m$.

Fissato $x \in \mathbb{R}^m$, esistono due scelte di $v \in \mathbb{R}^m$ che determinano una H_v tale per cui $H_v \cdot x$ è un multiplo di e_1 (primo vettore della base canonica); queste sono:

$$v = x \pm \|x\|_2 e_1.$$

In particolare, a meno di riorganizzare le operazioni, $v = (v_1, \dots, v_m)^T$ è uguale a x tranne per il primo elemento, che vale

$$v_1 = x_1 - \|x\|_2$$

oppure

$$v_1 = x_1 + \|x\|_2.$$

In una di queste, il denominatore soffre di errori di cancellazione; quale? (dipende dal segno di x_1 e da come scegliamo di calcolare v).

Esercizio 1. Scrivere una funzione `function v = householder_vector(x)` che calcoli v tale che H_v mandi x in un multiplo di e_1 evitando errori di cancellazione (cioè scegliendo il segno giusto).

La fattorizzazione QR

Data $A \in \mathbb{R}^{m \times n}$, con $m \geq n$, la fattorizzazione QR è una decomposizione $A = QR$ dove

- $Q \in \mathbb{R}^{m \times m}$ è una matrice ortogonale (i.e., $Q^T Q = I$).
- $R \in \mathbb{R}^{m \times n}$ è rettangolare e triangolare superiore.

Reminder: Per calcolare la (o, per essere più precisi, una) fattorizzazione QR di A si possono usare le matrici elementari di Householder seguendo un procedimento di triangolarizzazione simile a quello che avviene nell'eliminazione di Gauss. In particolare si applicano n trasformazioni di Householder a sinistra ($n - 1$ nel caso $m = n$) in modo da avere che

$$H_n \cdots H_1 \cdot A = R = \begin{bmatrix} * & \cdots & * \\ & \ddots & \vdots \\ & & * \end{bmatrix}$$

e $Q = H_1^T \cdots H_n^T$ è ortogonale.

Esercizio 2. Facendo uso della funzione `householder_vector`, implementata al punto precedente, scrivere una funzione `[Q,R] = my_qr(A)` che calcoli la fattorizzazione. Si testi la correttezza dell'algoritmo su matrici di propria scelta o generate casualmente. In particolare si verifichi che

- la fattorizzazione “ricostruisca” A , ovvero che

```
norm(A - Q * R)/norm(A)
```

restituisca un valore piccolo.

- La matrice Q sia ortogonale, ovvero che

```
norm(Q' * Q - eye(size(Q, 1)))
```

restituisca un valore piccolo.

- R sia triangolare superiore.

Problemi lineari ai minimi quadrati

A lezione abbiamo visto che il problema lineare ai minimi quadrati

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2 \tag{1}$$

può essere risolto con due strategie:

- risolvendo il sistema delle equazioni normali

$$A^T A x = A^T b$$

con il metodo dell'eliminazione di Gauss.

- Con il metodo QR per problemi ai minimi quadrati, ovvero risolvendo il sistema triangolare $R_1 x = c_1$ (di dimensione $n \times n$) in cui

$$A = QR, \quad R = \begin{bmatrix} R_1 \\ 0 \end{bmatrix}, \quad Q^T b = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}.$$

Esercizio 3. Si implementino le funzioni

```
function [x, res] = mq_normali(A, b)
function [x, res] = mq_qr(A, b)
```

che risolvono il problema (I) con le due strategie descritte sopra. I due metodi devono restituire sia la soluzione ottima x che il valore minimo del residuo. Si osservi che nel caso nel metodo QR, il valore ottimo del residuo corrisponde a $\|c_2\|_2$.

Si testino le due funzioni su problemi dove A e b sono generati casualmente, per dimensioni m ed n a scelta.

Esercizio 4. Si esegua il seguente codice per generare la matrice A ed il vettore b .

```
m = 10;
n = 5;
A = zeros(m,n);
for j = 1:n
    A = A + randn(m,1)*randn(1,n)*10^(-2*(j-1));
end
b = randn(m, 1);
```

Si risolva il problema ai minimi quadrati (I) con entrambi i metodi implementati al punto precedente e si confrontino le due soluzioni trovate. Quanto sono diverse? Quale dei due restituisce la soluzione migliore? Perché?

Laboratorio di Calcolo Numerico

Lezione 7

Malcondizionamento della matrice di Vandermonde

Dati $k + 1$ punti in $\mathbb{R}$, si definisce la matrice di Vandermonde V associata a tali punti come la matrice

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^k \\ 1 & x_1 & x_1^2 & \dots & x_1^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_k & x_k^2 & \dots & x_k^k \end{bmatrix}. \quad (1)$$

che sappiamo giocare un ruolo fondamentale nel problema di interpolazione sui nodi $x_0, \dots, x_k$.

Per costruire la matrice di Vandermonde esiste il comando `vander` che però restituisce le colonne in ordine diverso da quello in [\(1\)](#); per ottenere la matrice usuale usare

```
% x vettore di lunghezza k+1 contenente x0,...,xk
V = vander(x);
V = V(:,end:-1:1);
```

Esercizio 1. Prendere n nodi di interpolazione equispaziati nell'intervallo $[-2, 2]$, con n che varia da 2 a 20. Per ciascun n costruire la matrice di Vandermonde e salvarne il numero di condizionamento in un vettore. Fare un grafico `loglog` che riporti il numero di condizionamento della matrice in corrispondenza della quantità di nodi di interpolazione usati. Com'è la crescita del numero di condizionamento? (cioè quadratica, cubica, ... , esponenziale o altro?). **Suggerimento:** Per capire da un grafico in che modo cresce/decrese una quantità, si possono plottare nella stessa finestra delle curve di riferimento e vedere quale ha la crescita più simile. Ad esempio si può provare con le funzioni x, x^2, x^6, e^x .

Interpolazione nella base di Lagrange

Dati $k + 1$ punti di interpolazione

$$(x_0, y_0), \quad (x_1, y_1), \quad \dots \quad (x_k, y_k),$$

il polinomio interpolante di Lagrange è definito come

$$L_k(x) = \sum_{j=0}^k y_j \cdot \ell_j(x),$$

dove

$$\ell_j(x) = \prod_{i=0, i \neq j}^k \frac{x - x_i}{x_j - x_i}.$$

Esercizio 2. Si implementi una funzione Matlab

```
function w = valuta_lagrange(x, y, v)
```

che prende in ingresso

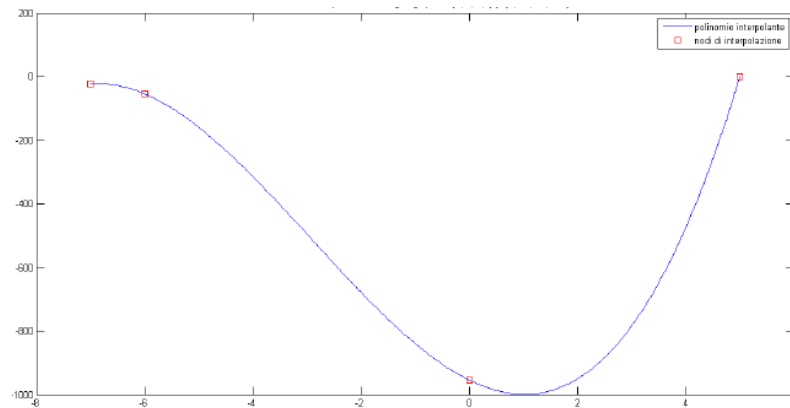
- due vettori x ed y , di lunghezza $k + 1$, rappresentanti i nodi dell'interpolazione ed i valori della funzione interpolata,
- un vettore v di lunghezza arbitraria, contenente i punti dove si vuole valutare il polinomio di interpolazione,

e restituisce il vettore w , di lunghezza uguale a quella di v , contenente le valutazioni del polinomio interpolante, di grado al più k , sui punti contenuti in v .

Per testare la correttezza dell'implementazione si utilizzi **valuta_lagrange** per valutare graficamente il polinomio interpolante di Lagrange $L_3(x)$ nei punti:

x	5	-7	-6	0
y	1	-23	-54	-954

Più precisamente, al fine di costruire il grafico di $L_3(x)$ e verificare che sia interpolante, si valuti su 1000 equispaziati nell'intervallo $[-7, 5]$ e si evidenzino sul grafico i nodi dell'interpolazione con un marcatore diverso. Si dovrebbe ottenere un grafico del tipo:



Errore di interpolazione

Data una funzione f e il suo polinomio interpolante P_k di grado al più k , costruito a partire dai nodi di interpolazione $x_0, x_1, \dots, x_k$, si può dimostrare che l'errore d'interpolazione che viene commesso in un certo punto x è pari a

$$E_k(x) = f(x) - P_k(x) = \frac{f^{(k+1)}(c_x)}{(k+1)!} \prod_{j=0}^k (x - x_j),$$

con c_x appartenente ad un intervallo contenente x ed i nodi di interpolazione. In particolare, se si vuole stimare l'errore d'interpolazione commesso su un intervallo $[a, b]$ contenente i nodi di interpolazione si può usare la seguente formula

$$\max_{x \in [a, b]} |E_k(x)| \leq \frac{\max_{x \in [a, b]} |f^{(k+1)}(x)|}{(k+1)!} \cdot \max_{x \in [a, b]} \prod_{j=0}^k |x - x_j|. \quad (2)$$

Esercizio 3. Si consideri la funzione

$$f(x) = \log(2+x), \quad x \in [-1, 1],$$

ed i polinomi di interpolazione $P_k(x)$ su nodi equispaziati, per $k = 3, 4$. Si calcolino numericamente le quantità (2) per $k = 3, 4$ (ad esempio per trovare i massimi della derivata e della produttoria si possono valutare su una griglia di punti su $[-1, 1]$ e poi prendere il massimo valore trovato). Infine si formino dei grafici in cui si verifica che $|E_k(x)|$ rimane sempre sotto le stime della quantità (2), trovate in precedenza.

Il fenomeno di Runge

Se si ha una funzione $f(x)$ continua in un intervallo $[a, b]$ ed in tale intervallo si calcolano polinomi di interpolazione di grado via via maggiore sembrerebbe naturale aspettarsi che la successione di tali polinomi converga uniformemente ad $f(x)$ in $[a, b]$. Ma nella realtà, per la maggior parte delle funzioni continue, ciò non è vero. Un esempio è fornito dalla funzione di Runge

$$f(x) = \frac{1}{1+x^2},$$

nell'intervallo $[-5, 5]$.

Esercizio 4. Costruire il grafico del polinomio interpolante, usando $n = 6, 7, \dots, 12$ nodi di interpolazione equispaziati nell'intervallo $[-5, 5]$, ottenuti valutando la funzione di Runge (usare la funzione `valuta_lagrange`). Riportare il grafico della funzione e dei polinomi interpolanti per alcuni valori di n . Cosa succede al crescere di n ?

Il fenomeno di Runge può essere evitato cambiando la scelta dei punti di interpolazione. Verifichiamo cosa succede se si sostituiscono i nodi equispaziati con i cosiddetti nodi di Chebyshev sull'intervallo $[a, b]$, definiti come:

$$\frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{(2j+1)\pi}{2k}\right), \quad j = 0, \dots, k-1.$$

Esercizio 5. Si ripeta l'esercizio precedente (costruzione del grafico dei polinomi interpolanti alla funzione di Runge), sostituendo gli n nodi equispaziati in $[-5, 5]$ con gli n nodi di Chebyshev associati allo stesso intervallo (si mantengano invece i nodi equispaziati per la costruzione del grafico).

Laboratorio di Calcolo Numerico

Lezione 8

Fitting di dati

Si scarichi dalla pagina elearn.ing il file `dati.mat` e si digiti

```
load dati.mat
```

che restituisce una matrice 1000×2 di coppie $(x, f(x))$ di una funzione sconosciuta $f(x)$ che cercheremo di approssimare nel senso dei minimi quadrati.

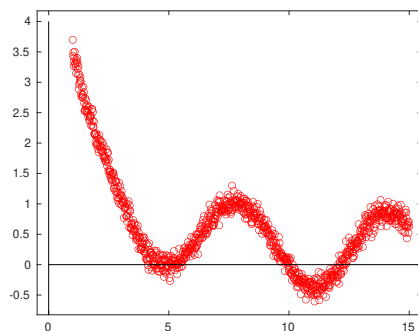


Figura 1: Dati contenuti nel file che trovate su e-learning

Esercizio 1. Si imposti il problema ai minimi quadrati corrispondente all'approssimazione di $f(x)$ con un polinomio di grado 6. Si risolva il problema tramite una delle funzioni implementate nella lezione di laboratorio 6 (`mq_qr` o `mq_normali`) e si mostri su un grafico la soluzione trovata insieme ai dati iniziali. Si ripeta l'esperimento considerando come approssimante una combinazione lineare delle seguenti funzioni modello

$$\exp(x), \quad \frac{1}{x}, \quad \sin(x).$$

Laboratorio di Calcolo Numerico

Lezione 9

Formule di Newton-Cotes generalizzate

L'idea delle formule di Newton-Cotes generalizzate (talvolta dette composite) è che per rendere l'approssimazione di $\int_a^b f(x)dx$ più accurata si può suddividere l'intervallo $[a, b]$ in L sottointervalli $[x_{i-1}, x_i]$ di uguale ampiezza e applicare una formula di Newton-Cotes in ciascuno di essi. (Attenzione alla notazione: L sottointervalli, $L + 1$ nodi x_j). Questo, nel caso della regola dei trapezi o della formula di Simpson da origine alle seguenti formule di quadratura:

- $J_1^{(G)}(f) = \frac{b-a}{2L} \left[f(x_0) + 2 \sum_{i=1}^{L-1} f(x_i) + f(x_L) \right]$
- $J_2^{(G)}(f) = \frac{b-a}{6L} \left[f(x_0) + 2 \sum_{i=1}^{L-1} f(x_i) + 4 \sum_{i=1}^L f\left(\frac{x_i+x_{i-1}}{2}\right) + f(x_L) \right]$

Esercizio 1. Si scriva una routine MATLAB

```
trapezi_gen(f, a, b, L)
```

che approssimi l'integrale di una funzione f (fornita in input come handle function) sull'intervallo $[a, b]$ usando la formula dei trapezi generalizzata con L sottointervalli.

- (i) Si testi la routine per il calcolo dell'integrale di un polinomio di grado 1. Come si capisce guardando alla formula dell'errore, il valore ottenuto per un polinomio di primo grado (cioè con derivata seconda nulla) deve essere esatto (a meno della precisione macchina). Confrontare quindi il valore ottenuto con quello esatto (calcolato a mano).

- (ii) Si approssimi il valore di

$$\int_1^2 \left(e^x + \frac{10}{x^2} \right) dx,$$

e si confronti il valore ottenuto con quello esatto (calcolabile a mano) o con quello restituito dalla funzione `integral` di MATLAB. In particolare, si tracci un grafico che mostri l'andamento del valore assoluto dell'errore al crescere del

numero di sottointervalli in cui si divide $[a, b]$. Rispetta le attese? Aggiungere alla figura il grafico della quantità

$$\max_{x \in [a, b]} |f''(x)| \cdot \frac{(b-a)^3}{12L^2}$$

che sappiamo essere una maggiorazione per il valore assoluto dell'errore.

Esercizio 2. Si scriva una routine MATLAB

```
simpson_gen(f, a, b, L)
```

che approssimi l'integrale di una funzione f (fornita in input come handle function) sull'intervallo $[a, b]$ usando la formula di Cavalieri-Simpson con L sottointervalli.

(i) Si testi la routine per il calcolo dell'integrale di un polinomio di grado 3, su cui dovrebbe fornire un risultato esatto (a meno della precisione di macchina).

(ii) Si approssimi il valore di

$$\int_1^2 \left(e^x + \frac{10}{x^2} \right) dx,$$

e si confronti il valore ottenuto con quello esatto (calcolabile a mano) o con quello restituito dalla funzione `integral` di MATLAB. In particolare, si tracci un grafico che mostri l'andamento del valore assoluto dell'errore al crescere del numero di sottointervalli in cui si divide $[a, b]$. Rispetta le attese? Aggiungere alla figura il grafico della quantità

$$\max_{x \in [a, b]} |f^{(4)}(x)| \cdot \frac{(b-a)^5}{2880L^4}$$

che sappiamo essere una maggiorazione per il valore assoluto dell'errore.

Contare il numero di valutazioni: trapezi vs Cavalieri Simpson

Abbiamo visto a lezione (e si vede facilmente dalle formule) che, a parità di sottointervalli L , la formula dei trapezi necessita di $L + 1$ valutazioni della funzione f , mentre la formula di Cavalieri-Simpson ne utilizza $2L + 1$. Vogliamo capire quale dei due metodi ha un costo minore (in termini di valutazioni di $f(x)$) se l'obiettivo è una data accuratezza sulla stima dell'integrale.

Esercizio 3. Si considerino sia il metodo dei trapezi generalizzato che la formula di Cavalieri-Simpson. Per $\epsilon = 10^{-1}, 10^{-2}, \dots, 10^{-7}$ si determini empiricamente/numericamente quale è il numero minimo di sottointervalli, $L_t(\epsilon)$ per i trapezi e $L_s(\epsilon)$ per Simpson, in cui deve essere suddiviso l'intervallo di integrazione affinché l'errore della stima dell'integrale

$$\int_0^3 x^2 \sin(x)^3 dx,$$

sia inferiore a ϵ . Su un grafico in scala logaritmica su entrambi gli assi, si mostrino le due curve rappresentanti il numero minimo di valutazioni (valutazioni, non sottointervalli) necessarie ai due metodi per raggiungere un errore minore di ϵ . Cosa si può concludere dal grafico ottenuto?

Laboratorio di Calcolo Numerico

Lezione 10

Metodi di punto fisso

Il metodo del punto fisso prevede di cercare lo zero di una funzione f attraverso la ricerca del punto fisso di un'altra funzione, chiamiamola ϕ , ottenuta manipolando f . Se l'equazione $f(x) = 0$ è riscritta nella forma $x = \phi(x)$, è possibile utilizzare un metodo iterativo di punto fisso della seguente forma

$$\begin{cases} x_0 \text{ dato} \\ x_{n+1} = \phi(x_n) \end{cases}, \quad n = 0, 1, 2, \dots, \quad (1)$$

e sperare che la successione $\{x_n\}_{n \in \mathbb{N}}$ converga ad una radice di f . Partendo da una certa funzione f si possono derivare infinite funzioni ϕ che sono la riscrittura di f nella forma di punto fisso, ma non è detto che ognuna di queste funzioni porti ad una successione convergente.

Esercizio 1. Si implementi una funzione Matlab

```
function xvec = puntofisso(phi, x0, tol, maxit)
```

che calcoli la successione $\{x_n\}$ (restituita nel vettore **xvec**) per una funzione $\phi(x)$ passata come handle function, un punto di partenza **x0** e per un massimo numero di iterazioni **maxit**. Si faccia terminare l'esecuzione prematuramente (cioè prima di aver eseguito **maxit** iterazioni) nel caso si verifichi $|x_n - \phi(x_n)| < \text{tol}$.

Esercizio 2. Si consideri l'equazione $f(x) = 0$ con

$$f(x) = x - x^{\frac{1}{3}} - 2,$$

- Mostrare teoricamente che $f(x)$ ha un unico zero e si trova nell'intervallo $[3, 5]$.
- Utilizzare, mediante la funzione **puntofisso**, le seguenti 3 iterazioni di punto fisso (dopo aver verificato che si ottengono davvero manipolando $f(x)$) per determinare lo zero della funzione, impiegando come punto iniziale $x_0 = 3$ (e valori **tol** e **maxit** di vostra scelta):

$$\begin{aligned} \phi_1(x) &= x^{\frac{1}{3}} + 2, \\ \phi_2(x) &= (x - 2)^3, \end{aligned}$$

$$\phi_3(x) = \frac{6+2x^{\frac{1}{3}}}{3-x^{-\frac{2}{3}}}.$$

- Si stimi l'errore relativo dei tre metodi nel seguente modo. Si utilizzi la funzione interna di Matlab `fzero` per ottenere un valore di riferimento α della radice che si sta cercando. Quindi si faccia uso di α e degli elementi del vettore `xvec` per calcolare la successione degli errori relativi $\frac{|x_n - \alpha|}{|\alpha|}$ (per ognuna delle tre successioni). Infine traccino dei grafici, con scala logaritmica sull'asse delle y , contenenti l'andamento dell'errore relativo vs le iterazioni, per i 3 diversi metodi.
- Giustificare la mancata convergenza dell'iterazione di punto fisso collegata a $\phi_2(x)$ e della convergenza negli altri 2 casi.

Metodo di Newton

Il metodo di Newton (detto anche metodo delle tangenti) è un particolare metodo stazionario ad un punto, che cerca di approssimare una radice di una funzione derivabile $f(x)$ generando la successione:

$$\begin{cases} x_0 \text{ dato} \\ x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \end{cases}, \quad n = 0, 1, 2, \dots$$

Esercizio 3. Si implementi una funzione

```
function [xvec, valvec] = newton(f, df, x0, tol, maxit)
```

che esegua `maxit` passi del metodo di Newton per la funzione `f`, con derivata `df`, a partire dal punto `x0`. Il metodo termina prematuramente se si verifica la condizione $|f(x_n)| < \text{tol}$. Infine si restituiscano i vettori `xvec` e `valvec` contenenti i punti x_n generati dal metodo di Newton e le corrispondenti valutazioni di f , in modo da poter valutare la convergenza.

Esercizio 4. Si applichi il metodo di Newton al calcolo degli zeri della funzione

$$f(x) = x^3 + 4x \cos(x) - 2,$$

nell'intervallo $[0, 2]$. Si testino vari punti di partenza x_0 e si traccino dei grafici che riportino l'andamento delle valutazioni (contenute in `valvec`) e dell'errore relativo. Per il calcolo di quest'ultimo si utilizzi il valore della radice α stimato con la funzione `fzero`.

Il metodo di Newton potrebbe non convergere

Esercizio 5. Si utilizzi il metodo di Newton per il calcolo della radice quadrata x di un numero positivo A , inteso come zero della funzione $f(x) = x^2 - A$. Si faccia partire l'algoritmo da un punto iniziale a scelta e si riprovi poi con il suo opposto, e poi con $x = 0$. Commentare i risultati alla luce della convergenza o meno del metodo e del valore a cui converge.

LEZIONE 1

SOLVE2.M

```
function [x1, x2] = solve2(a, b, c)
if b>0
    x1 = (-b-sqrt(b^2-4*a*c))/(2*a);
else
    x1 = (-b+sqrt(b^2-4*a*c))/(2*a);
end
x2 = c/(a*x1);
end
```

MYEXP.M

```
function a = myexp(x,n)
    % calcola exp(x) con la serie di Taylor troncata all'n-esimo
    termine
    a = 1; %accumulatore
    for k = 1:n
        a = a + x^k/factorial(k);
    end
end
```

MYEXP2.M

```
function a = myexp2(x,n)
    % calcola e^x con Taylor troncato
    % ma usa solo O(n) operazioni ed evita overflow
    t = 1; % accumulatore che contiene il termine generico della
    sommatoria
    a = 1; % accumulatore che contiene le somme parziali
    for k = 1:n
        t = t * x / k;
        a = a + t;
    end
end
```

LEZIONE 2

MYDET.M

```
function d = mydet(A)
    n = size(A, 1);
    if n == 1
        d = A;
    elseif n == 2
        d = A(1,1)*A(2,2)-A(1,2)*A(2,1);
    else
        d = 0;
        for j=1:n
            d = d + (-1)^(j+1) * A(1, j) * mydet(A(2:n, [1:j-1,
j+1:n])));
        end
    end
end
```

end

LEZIONE 3

DISEGNAGERSH.M

```
function disegna_gersh(A)
close
% funzione che disegna i cerchi di Gershgorin di una matrice
    n = size(A, 1);
    c = diag(A); % vettore contenente i centri
    r = abs(A - diag(c)) * ones(n, 1); % vettore contenente i raggi
    figure
    hold on
    for j = 1:n
        circle(c(j), r(j), 'b');
    end
    alpha(0.15);
    l = eig(A);
    pp = plot(real(l), imag(l), 'rx', real(c), imag(c), 'ko');
    axis equal;
    legend(pp, 'autovalori', 'centri');

end

function circle(c, r, col)
    t = linspace(0, 2*pi, 1000);
    x = r * cos(t);
    y = r * sin(t);
    patch(real(c) + x, imag(c) + y, col, 'EdgeColor', col);
end
```

DISEGNAGERSH2.M

```
function disegna_gersh2(A)
close
% funzione che disegna i cerchi di Gershgorin di una matrice e della
sua trasposta
    n = size(A, 1);
    c = diag(A); % vettore contenente i centri
    r = abs(A - diag(c)) * ones(n, 1); % vettore contenente i raggi
associati alla matrice
    rt = ones(1, n) * abs(A - diag(c)); % vettore contenente i raggi
associati alla matrice trasposta
    figure
    hold on
    for j = 1:n
        circle(c(j), r(j), 'y');
        circle(c(j), rt(j), 'b');
    end
    alpha(.15);
    l = eig(A);
    pp = plot(real(l), imag(l), 'rx', real(c), imag(c), 'ko');
    axis equal;
```

```

        legend(pp, 'autovalori', 'centri');

end
function circle(c, r, col)
    t = linspace(0, 2*pi, 1000);
    x = r * cos(t);
    y = r * sin(t);
    patch(real(c) + x, imag(c) + y, col, 'EdgeColor', col);
end

```

LEZIONE 4

INFSOLVE.M

```

function x=inf_solve(L,b)
s=size(L);
n=s(1);
x=zeros(n,1);
    for i=1:n
        p=b(i); % accumulatore
        for j=1:i-1
            p=p-L(i,j)*x(j);
        end
        x(i)=p/L(i,i);
    end
end

```

INFSOLVE2.M

```

function x=inf_solve(L,b)
s=size(L);
n=s(1);
x=zeros(n,1);
    for i=1:n
        p=b(i); % accumulatore
        p = p-L(i, 1:i-1)*x(1:i-1);
        x(i)=p/L(i,i);
    end
end

```

SUPSOLVE.M

```

function x=sup_solve(U,b)
s=size(U);
n=s(1);
x=zeros(n,1);
    for i=n:-1:1
        p=b(i); % accumulatore
        for j=i+1:n
            p=p-U(i,j)*x(j);
        end
        x(i)=p/U(i,i);
    end
end

```

SUPSOLVE2.M

```
function x=sup_solve2(U,b)
s=size(U);
n=s(1);
x=zeros(n,1);
    for i=n:-1:1
        p=b(i); % accumulatore
        p = p - U(i, i+1:n) * x(i+1:n);
        x(i)=p/U(i,i);
    end
end
```

MYGAUSS.M

```
function [U, c] = my_gauss(A, b)
n = size(A, 1);
Ab = [A, b]; % costruisco la matrice aumentata
for j=1:n-1
    for i=j+1:n
        lij = Ab(i, j)/Ab(j, j); % moltiplicatore (i,j)
        Ab(i, j:n+1) = Ab(i, j:n+1) - lij * Ab(j, j:n+1);
    end
    Ab(j+1:n, j) = 0; % Metto a zero la parte sotto la diagonale
    della colonna j
end
U = Ab(:, 1:n);
c = Ab(:, n+1);
end
```

SYSSOLVE.M

```
function x = x = sys_solve(A,b)
[U, c] = my_gauss(A, b);
x = sup_solve(U, c);
end
```

LEZIONE 5

MYDET2.M

```
function d = mydet2(A)
n = size(A, 1);
for j=1:n-1
    if A(j, j) == 0
        error('ho incontrato un pivot nullo')
    end
    for i=j+1:n
        lij = A(i, j)/A(j, j); % moltiplicatore (i,j)
        A(i, j:end) = A(i, j:end) - lij * A(j, j:end);
    end
end
d = prod(diag(A)); % Estraggo la diagonale di A con diag, e faccio il
prodotto di tutti gli elementi con prod
```



```
end
```

TESTDETERMINANT.M

```
n = [1:150];
tempi= zeros(n(end),3);
for j = 1:n(end)
    A = randn(j);
    tempi(j, 1) = timeit(@() det(A));
    tempi(j, 2) = timeit(@() mydet2(A));
    if j <= 10
        tempi(j, 3) = timeit(@() mydet(A));
    end
end
loglog(n, n.^3, 'k')
hold on
loglog(n, tempi(:, 1), 'y', n, tempi(:, 2), 'r', n, tempi(:, 3), 'b')
legend('n^3', 'det', 'mydet2', 'mydet')
```

GAUSSSEIDEL.M

```
function x = gaussseidel(A, b, k)
%GAUSSSEIDEL Soluzione di un sistema lineare usando il metodo di
Gauss-Seidel.
%
% X = GAUSSSEIDEL(A, B, K) risolve il sistema lineare AX = B usando il
metodo di
% Gauss-Seidel sulla matrice A. Vengono effettuate esattamente K
iterazioni.

m = size(A, 1);
n = size(A, 2);

if m ~= n
    error('La matrice A non è quadrata');
end

if size(b, 1) ~= n
    error('Il vettore b non ha dimensioni compatibili con A');
end

x = b;

for j = 1 : k
    % Iterazione j

    for i = 1 : n
        % Questa e le istruzioni seguenti sono scritte per funzionare
anche
        % quando b ha più colonne, e dunque vengono risolti più sistemi
        % lineari in un colpo solo.
        x(i, :) = b(i, :);
```

```

        for s = 1 : n
            if s ~= i
                x(i, :) = x(i, :) - A(i, s) * x(s, :);
            end
        end
        x(i, :) = A(i,i) \ x(i, :);
    end

    % Stampiamo il residuo
    norm(A * x - b)
end

```

JACOBI.M

```

function x = jacobi(A, b, k)
%JACOBI Soluzione di un sistema lineare usando il metodo di Jacobi.
%
% X = JACOBI(A, B, K) risolve il sistema lineare AX = B usando il
metodo di
%      Jacobi sulla matrice A. Vengono effettuate esattamente K
iterazioni.

m = size(A, 1);
n = size(A, 2);

if m ~= n
    error('La matrice A non è quadrata');
end

if size(b, 1) ~= n
    error('Il vettore b non ha dimensioni compatibili con A');
end

x = b;
x_new = zeros(n,1);
for j = 1 : k
    % Iterazione j

    for i = 1 : n
        % Questa e le istruzioni seguenti sono scritte per funzionare
anche
        % quando b ha più colonne, e dunque vengono risolti più sistemi
        % lineari in un colpo solo.
        S = 0;

        for s = 1 : n
            if s ~= i
                S = S + A(i, s) * x(s, :);
            end
        end
        x_new(i) = (b(i) - S) / A(i,i);
    end
end

```

```

        x_new(i, :) = (b(i, :)-S)/A(i,i);
    end

    x = x_new;

    % Stampiamo il residuo
    norm(A * x - b)
end

```

LEZIONE 6

HOUSEHOLDER.M

```

function v = householder_vector(x)
    nrm = norm(x);
    if nrm == 0
        v = 0;
    else
        v = x;
        v(1) = v(1) + sign(x(1)) * nrm;
    end
end

```

MYQR.M

```

function [Q, R] = my_qr(A)
%MY_QR Calcola la fattorizzazione QR di A.
%
% [Q, R] = MY_QR(A) calcola la fattorizzazione QR di A, utilizzando
%   riflettori di Householder.
%
% Questa implementazione funziona anche con matrici rettangolari.

[m, n] = size(A);

R = A;
Q = eye(m);

% Riduciamo tutte le colonne in modo che siano triangolari superiori.
% Dobbiamo occuparci delle prime min(n, m - 1) colonne.
for j = 1 : min(m - 1, n)
    % Calcolo un riflettore di Householder che mappa la parte restante
    % della colonne j-esima di R in un multiplo di e1.
    u = householder_vector(R(j:end, j));
    b = 2/norm(u)^2;
    % Lo applico ad R a sx, e a Q a dx.
    R(j:end,j:end) = R(j:end,j:end) - b * u * (u' * R(j:end,j:end));
    Q(:,j:end) = Q(:,j:end) - b * (Q(:,j:end) * u) * u';

    % Dato che R è triangolare superiore, la parte triangolare
    inferiore si

```

```

        % può impostare a zero manualmente, per evitare di vedere del
round-off
        R(j+1:end,j) = 0;
end

```

```

end

```

MQQR.M

```

function [x, res] = mq_qr(A, b)
    [m, n] = size(A);
    [Q, R] = my_qr(A);
    c = Q'*b;
    x = R(1:n, 1:n) \ c(1:n);
    res = norm(c(n+1:end));
end

```

MQNORMALI.M

```

function [x, res] = mq_normali(A, b)
    AA = A'*A;
    Ab = A'*b;
    x = AA\Ab;
    res = norm(b-A*x);
end

```

LEZIONE 7

TESTVANDERMONDE.M

```

i = 1;
for n =2:20
    x = linspace(-2, 2, n);
    V = vander(x);
    V=V(:, end:-1:1);
    condiz(i)=cond(V);
    i=i+1;
end
punti = [2:n];
loglog(punti, condiz, '-m*')
hold on
loglog(punti, punti, 'r', punti, punti.^2, 'b', punti, punti.^6, 'c',
punti, exp(punti), 'k')
legend('cond(V)', 'n', 'n^2', 'n^6', 'exp(x)')

```

VALUTALAGRANGE.M

```

function w = valuta_lagrange(x, y, v)
    k = length(x);
    m = length(v);
    w = zeros(m, 1);
    % rendo i tre vettori dei vettori colonna

```

```

x = x(:);
y = y(:);
v = v(:)
for j = 1:k
    xx = x([1:j-1, j+1:end]); % faccio un vettore temporaneo con
i nodi di interpolazione meno il j-esimo
    tmp = ones(m, 1); % accumulatore
    for h = 1:k-1
        tmp = tmp .* (v - xx(h));
    end
    tmp = tmp / prod(x(j) - xx);
    w = w + y(j) * tmp;
end
end

```

ESEMPIORUNGE.M

```

function esempio_runge(k)
f = @(x) (100*x.^2-100*x + 26).^(-1);
s = linspace(0, 1, 1e3);
plot(s, f(s), 'b')
hold on
for j = 3:k
    d = linspace(0, 1, j);
    fd = f(d);
    px = plot(d, fd, 'mo', 'MarkerFaceColor', 'm', 'MarkerSize', 6);
    pip = plot(s, valuta_lagrange(d, fd, s), 'r');
    st = sprintf('Polinomio interpolante di grado %d', j);
    leg = legend('Funzione di Runge', st);
    axis([0, 1, -.5, 2.5])
    pause
    if j < k
        delete(px);
        delete(pip);
    end
end
end

axis([0, 1, -.2, 2.5])

end

```

LEZIONE 9

TRAPEZIGEN.M

```

function y = trapezi_gen(f, a, b, L)
x = linspace(a, b, L+1);
y = 0;
for j = 1:L+1
    if j>1 && j<L+1
        y = y + 2*f(x(j));
    else
        y = y + f(x(j));
    end
end

```

```

        end
    end
    y = (b-a)*y/(2*L);
end

```

SIMPSONGEN.M

```

function y = simpson_gen(f, a, b, L)
    x = linspace(a, b, 2*L+1);
    y = 0;
    for j = 1:2*L+1
        if j==1 || j== 2*L+1
            y = y + f(x(j));
        elseif mod(j, 2) == 1
            y = y + 2*f(x(j));
        else
            y = y + 4*f(x(j));
        end
    end
    y = (b-a)*y/(6*L);
end

```

TRAPEZIVSSIMPSON.M

```

f = @(t) t.^2.*cos(t).^3;
true_val = integral(f, 0, 3);
tol = 10.^[-1:-1:-7];
L_trap = zeros(length(tol), 1);
L_simp = zeros(length(tol), 1);
% Loop per i trapezi
L = 1;
it = 1;
while 1
    app = trapezi_gen(f, 0, 3, L);
    if abs(app-true_val) < tol(end)
        L_trap(end) = L;
        break
    else
        if abs(app-true_val) < tol(it)
            L_trap(it) = L;
            it = it+1;
        end
        L = L + 1;
    end
end

% Loop per simpson
L = 1;
it = 1;
while 1
    app = simpson_gen(f, 0, 3, L);
    if abs(app-true_val) < tol(end)

```

```

        L_simp(end) = L;
        break
    else
        if abs(app-true_val) < tol(it)
            L_simp(it) = L;
            it = it+1;
        end
        L = L + 1;
    end
end

loglog(tol, L_trap+1, 'r', tol, 2*L_simp+1, 'b')
legend('trapezi', 'simpson')
xlabel('errore assoluto')
ylabel('valutazioni della funzione')
title('Trapezi vs Simpson')

```

LEZIONE 10

PUNTOFISSO.M

```

function xvec = puntofisso(phi, x0, tol, maxit)
    xvec = x0;
    for j = 1:maxit
        xnew = phi(xvec(end));
        xvec = [xvec; xnew];
        if abs(xvec(end)-xvec(end-1)) < tol
            break
        end
    end
end

```

NEWTON.M

```

function [xvec, valvec] = newton(f, df, x0, tol, maxit)
    xvec = x0;
    valvec = f(x0);
    for j = 1:maxit
        xnew = xvec(end) - valvec(end)/df(xvec(end));
        xvec = [xvec; xnew];
        valvec = [valvec; f(xnew)];
        if abs(valvec(end)) < tol
            break
        end
    end
end

```